R1.02 — Développement d'interfaces web : Architecture Client-Serveur et Protocole HTTP

Thème principal : Fonctionnement du Web, Architecture Client-Serveur et HTTP

Synthèse exécutive : Ce cours pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur asymétrique et le protocole HTTP. Il explique comment un navigateur (client) formule des requêtes standardisées pour obtenir des ressources auprès d'un serveur web, comment ce dernier y répond de manière asynchrone et sans état (stateless), et comment décoder les codes de statut HTTP pour diagnostiquer et déboguer efficacement des applications web.

L'Architecture Client-Serveur et le Principe Stateless

L'architecture client-serveur est le modèle standard de communication sur le Web, établissant une distinction stricte et asymétrique des rôles. Le client (ex: Firefox, Chrome) prend toujours l'initiative en envoyant un message structuré (la requête HTTP) à travers le réseau. Le serveur reste dans un état d'écoute passive, traite la demande (en exécutant éventuellement des scripts ou en interrogeant une base de données), puis renvoie une réponse. Une caractéristique fondamentale du protocole HTTP est d'être 'stateless' (sans état) : chaque transaction requête-réponse est totalement isolée et autonome. Le serveur oublie instantanément le client dès que la réponse est transmise. Si cette conception simplifie la scalabilité et allège la charge des serveurs, elle nécessite l'emploi de mécanismes de contournement comme les cookies et les sessions pour maintenir des contextes utilisateurs (comme un panier d'achat ou une session connectée).

- Client (Navigateur) : Logiciel actif qui initie la communication en envoyant une requête.
- Serveur Web : Processus logiciel passif qui écoute sur un port (ex: 80 ou 443) et attend les requêtes.
- Protocole Stateless (Sans état) : Absence de mémoire des requêtes précédentes par le protocole HTTP.

Cas concret enseignant : *Le professeur explique que lorsqu'un étudiant se connecte sur le site de l'IUT de Bayonne, son navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque image, feuille de style CSS et script JavaScript associé. Il compare également HTTP à un guichetier amnésique qui traite chaque visiteur comme un inconnu, exigeant un badge (cookie de session) à chaque interaction.*

Structure des Messages HTTP et Méthodes (GET vs POST)

Un message HTTP est purement textuel et structuré de manière rigoureuse. La requête comprend une ligne de requête (méthode, URI, version), des en-têtes (headers) fournissant des métadonnées essentielles comme le domaine cible via 'Host' ou le type de client via 'User-Agent', une ligne vide obligatoire, et un corps (body) optionnel. La réponse du serveur adopte une structure similaire mais commence par une ligne de statut. Le choix de la méthode est primordial : GET transmet les données dans l'URL avec des limites de taille (environ 2048 caractères), tandis que POST encapsule les données dans le corps de la requête. Attention, POST n'est pas sécurisé par défaut sans chiffrement HTTPS.

- Ligne de requête / statut : Contient la méthode, l'URI et la version du protocole pour la requête, ou le code de statut et sa description pour la réponse.
- En-têtes (Headers) : Métadonnées sous forme de paires clé-valeur décrivant le contexte (Host, User-Agent, Content-Type).
- Méthode GET : Conçue pour la consultation, paramètres visibles dans l'URL, sûre et idempotente.
- Méthode POST : Conçue pour modifier l'état du serveur, paramètres encapsulés de manière invisible dans le corps (body).

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est purement textuel et en clair par défaut. Utiliser la méthode GET pour transmettre un mot de passe constitue une faille de sécurité majeure (connue sous le nom de 'shoulder surfing'), car les données apparaissent en clair dans l'historique du navigateur ou à la vue d'une personne se trouvant derrière l'utilisateur.*

La Classification des Codes de Statut HTTP

Chaque réponse HTTP commence par un code numérique à trois chiffres permettant de classifier immédiatement le résultat. Il est crucial pour un développeur de distinguer les erreurs de la famille 4xx, qui incombent au client (mauvaise URL, syntaxe incorrecte, absence d'autorisation), des erreurs de la famille 5xx, qui révèlent un dysfonctionnement interne de l'infrastructure ou du code serveur (script planté, base de données inaccessible), nécessitant une intervention technique directe.

- Codes 2xx (Succès) : La requête a été reçue et traitée avec succès (ex: 200 OK).
- Codes 3xx (Redirection) : Actions supplémentaires requises par le client (ex: 301 Moved Permanently).
- Codes 4xx (Erreur Client) : Anomalie dans la requête du client, URL erronée (ex: 404 Not Found, 403 Forbidden).
- Codes 5xx (Erreur Serveur) : Défaillance interne du serveur malgré une requête valide (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs paniquaient en pensant que le site était piraté alors qu'il s'agissait simplement d'un point-virgule manquant dans son script PHP.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Avec GET, les identifiants et mots de passe s'afficheraient en clair directement dans l'URL (ex: ?user=mat&pass;=1234), ce qui les rend visibles dans l'historique et les logs. Avec POST, les données voyagent cachées dans le corps (body) de la requête HTTP, invisible dans la barre d'adresse.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyser la première ligne : 'GET /images/logo.png HTTP/1.1'. Le premier mot 'GET' est la méthode HTTP utilisée pour récupérer une ressource sans modifier l'état du serveur. L'URI cible est '/images/logo.png' et la version du protocole est 'HTTP/1.1'. 2. Identifier l'hôte cible en lisant l'en-tête 'Host: iutbayonne.univ-pau.fr', qui spécifie le domaine cible, indispensable pour le virtual hosting. 3. Analyser l'agent utilisateur : 'User-Agent: Mozilla/5.0' identifie le navigateur client. 4. Scénario d'absence de ressource : Si le fichier 'logo.png' n'existe pas dans le répertoire du serveur web, celui-ci ne peut pas honorer la demande et renvoie un code de statut de la classe 4xx, précisément le code '404 Not Found'.

Résultat : Méthode : GET, Ressource : '/images/logo.png', Hôte cible : 'iutbayonne.univ-pau.fr', Code de retour en cas d'absence : 404 Not Found (Erreur Client).

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : C'est un protocole sans état où chaque requête est indépendante, et le serveur ne conserve aucune mémoire des requêtes précédentes.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les codes 4xx indiquent une erreur provenant du client (ex: mauvaise URL), tandis que les codes 5xx indiquent une défaillance interne du serveur.

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que GET expose les paramètres dans l'URL en clair, ce qui compromet la sécurité (historique, logs, affichage à l'écran).